On Job Training-01

1.Select any one job role of his/her interest in an organization for next 4 weeks and submit a report to the training supervisor and copy to the cohort owner focusing on:

Job Role: AIML and Software Development Intern
Responsibility: Development
Phase: Core implementation

In recent years, the use of machine learning in healthcare has significantly increased due to its ability to analyse large volumes of complex data and provide accurate predictions. Genetic diseases, in particular, require early detection and analysis as they are often influenced by multiple factors such as family history, gene mutations and environmental conditions. Traditional methods are time consuming and lack accuracy.

This project aims to develop a machine learning-based web application for predicting the risk of genetic diseases using patient data and family history. The system integrates front-end user interfaces, backend processing, cloud database management, and machine learning components to create a complete prediction pipeline.

The work carried out during the internship involved designing user authentication systems, developing dynamic dashboards, implementing dataset handling and pre-processing techniques, and integrating machine learning functionalities into a web-based platform. The objective was to build a scalable and user-friendly system for users with minimal technical knowledge that can assist in early risk prediction of genetic disorders.

i) Application of Skills and Technical Knowledge

During the internship, a wide range of technical skills and tools were applied to develop the system. The project required integration of software development concepts, machine learning techniques, and web technologies.

- ✓ **Programming Language:** Python

Python was primarily used for backend logic development in my project along with API creation, data handling and integration.

- ✓ **Libraries Used:** Pandas, Scikit-learn

For data processing, Pandas library was used to clean, pre-process and structure the dataset (Handling missing values, removing duplicates, encoding categorical variables and normalization)
For Machine learning preparation Scikit library was used for training and evaluation.

- ✓ **Backend Framework:** Flask

The flask framework was utilized to design backend services and REST APIs. These API's handles user authentication and communication between the front-end and machine learning
modules, along with efficient routing and request handling.

- ✓ **Frontend Technologies:** Tkinter, HTML, CSS, JavaScript

Tkinter was used to design GUI based login and signup interfaces while HTML, CSS and JS were used to create web based dashboard and prediction pages. Javascript was specifically used to implement dynamic visualizations.

- ✓ **Database:** Firebase (Authentication + Realtime DB)

For database management, firebase was integrated as a cloud based real time database used to store user credentials and personal details securely. Real time was used for dynamic data storage and retrieval

- ✓ **Testing Tool:** Postman

Postman was extensively used for API testing and validation. Each backend endpoint was tested independently to verify before integration with frontend

- ✓ **Development Tools:** PyCharm, Spyder, Sublime Text

PyCharm and Sublime was used for Frontend and Spyder was used for Backend, with python being the main primary coding language.

The integration of these technologies required a strong understanding of both software development and machine learning principles. The use of Flask enabled efficient communication between the frontend and backend, while Firebase provided a reliable cloud-based solution for storing and retrieving user data.

Data handling was a critical aspect of the project, where libraries such as Pandas were used for data cleaning, transformation, and analysis. Additionally, frontend technologies such as HTML, CSS, and JavaScript were used to design intuitive interfaces and implement dynamic visualizations, enhancing the overall user experience

ii) Performance on Assigned Tasks and Projects

The assigned tasks were completed in a structured and progressive manner over the internship period. The development process followed a modular approach, where each component of the system was built and tested independently before integration. This approach ensured that errors were identified early and resolved efficiently.

Regular testing using tools like Postman helped in validating API responses and ensuring seamless communication between different components of the system. Debugging played a crucial role throughout the development process, especially while handling real-world dataset challenges such as inconsistent data formats and missing values.

- Developed **user authentication system** (login, signup, CAPTCHA)

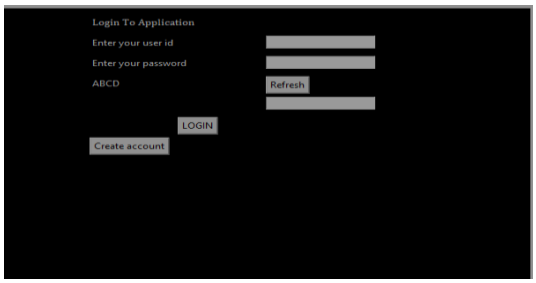


Figure 2.1.1: Initial Login Page (Before UI Enhancement)

- Integrated **Firebase database for real-time data storage**

I developed a complete user authentication system including account creation, login validation, CAPTCHA verification, and secure storage of user data using Firebase. This ensured that only authorized users could access the system.

- Built **dynamic dashboard displaying user details**
- Implemented **update and modification functionality**

I implemented dynamic front-end interfaces including login, signup, dashboard, and machine learning prediction pages. These interfaces were designed to be interactive and user-friendly, with proper navigation between different modules

- Designed **ML prediction interface**
- Integrated **backend and frontend communication**

I also worked on backend integration, where front-end inputs were successfully connected to backend APIs using HTTP requests. This enabled real-time communication between the user interface and server-side logic.

- Created **dataset upload and pre-processing module**
- Implemented **data cleaning (missing values, duplicates, formatting)**

In addition, I handled dataset processing tasks, including reading Excel files, cleaning data, handling missing values, removing duplicates, and converting data into structured formats. These steps were crucial for preparing the dataset for machine learning operations.

- Developed **data visualization (donut chart & risk gauge meter)**

I also contributed to the visualization component by implementing a dynamic risk gauge meter, which provides a clear graphical representation of prediction results. This improved the interpretability of the system output.

- Perfomed **API testing using Postman**
- Debugged and resolved system errors

I encountered several technical challenges such as API errors,data format inconsistencies, and UI alignment issues. These were resolved through debugging, testing, and iterative
improvements, which enhanced the overall system stability. All tasks were completed within the given timeline, demonstrating consistency, problem solving ability and adaptability.

iii) Value Addition to the Organization

The internship contributed meaningful value to the organization by demonstrating how machine learning can be practically applied in the healthcare domain to address real-world challenges in genetic disease detection.

- The development of a structured data pre-processing pipeline added direct operational value, as it enables the system to handle inconsistent, real-world datasets automatically — reducing manual effort and improving data reliability before model training.
- From a usability standpoint, the integration of a dynamic risk gauge meter and intuitive interface made the system accessible to non-technical users such as doctors and researchers, bridging the gap between complex ML outputs and practical healthcare decision-making.
- A key analytical contribution was the identification of system limitations — particularly the dependency on user-uploaded datasets — and the proposal of a pre-trained model integration as a future improvement. This shows an understanding of deployment readiness beyond just development.

Finally, optimizing the system output to display only relevant metadata rather than raw data improved both performance and user experience, making the application leaner and more professional. Overall, the project demonstrated that predictive ML systems, when designed thoughtfully, can serve as effective decision-support tools in preventive healthcare.

iv) Job Role Understanding and Relevance

The role undertaken during the internship was aligned with that of a Machine Learning and Software Development Intern.

The responsibilities included both developmental and analytical tasks

- Data pre-processing and validation
- Backend API development
- Frontend interface design
- Machine learning integration
- Debugging and performance optimization

It helped understanding how theoretical concepts learned can be applied to real-world.

. Use Case-01

2. Document a Use Case on a Task where he/she is working as an intern

Use Case Documentation

2.1 Use Case Title

Genetic Disease Risk Prediction System

2.2 Description

The system allows users to input patient data or upload datasets to predict the risk of genetic diseases using machine learning techniques. It provides both numerical and visual outputs for better understanding.

2.3 Actors

- User (Doctor / Researcher / Admin)
- System

2.4 Inputs

- Age

Certain genetic conditions are prone to occur in adults and may sometimes not manifest at all

- Gender

Some genetic disorders are sex and chromosome linked

- SNP values

Single Nucleotide Polymorphism values represent variations in genetic sequences which can play a crucial role in mutations

- Gene expression values

Indicates activity levels of specific genes. Abnormal gene expression patterns can be strong indicators of potential genetic disorders

- Family history

Genetic diseases are often inherited, this helps the system identify hereditary risk patterns based on previous generations.

- Dataset upload (CSV/Excel)

2.5 Process

1. System Execution

```
C:\Users\HOME\OneDrive\Desktop\fullstackmlproject\backend>py api.py
 * Serving Flask app 'api'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

```
C:\Users\HOME\OneDrive\Desktop\fullstackmlproject\frontend>py application.py
{"Contact":"9141208939","Email_Id":"gmedhau@gmail.com","First_name":"Medha","Last_name":"Cassano","User_Id":"divyaaaa"}
```

Figure 2.2.1: Backend and Frontend initialization of the system

2. Authentication

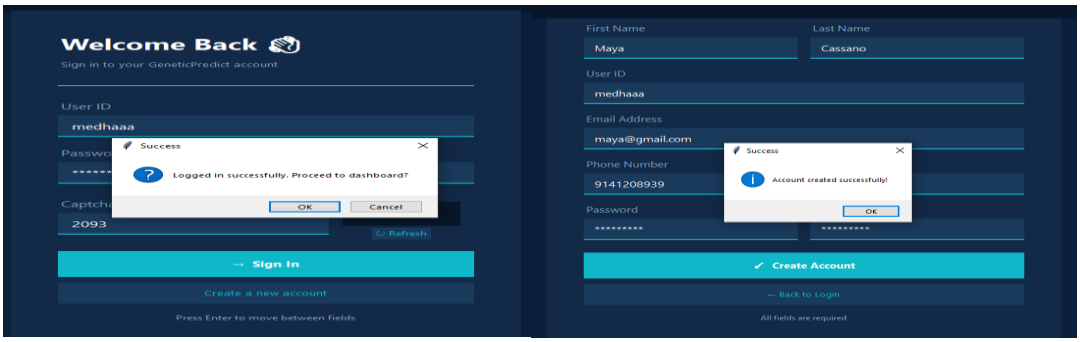


Figure 2.2.2: Login and Signup page
Create your account and login using your unique userid and password

3. Dashboard Page

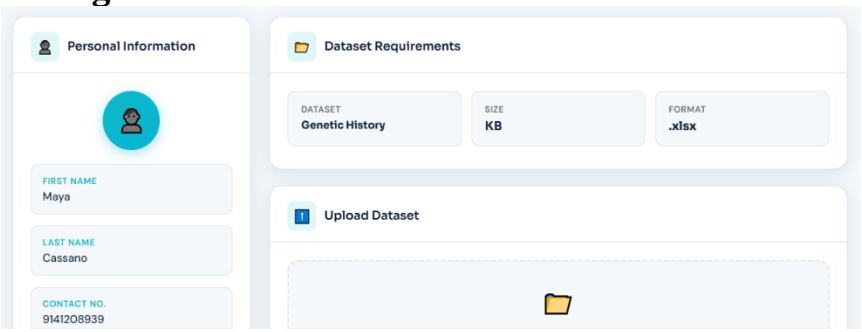


Figure 2.2.3: Dashboard page

Your dashboard which consists of your personal details on the left side and functional information (dataset uploading, properties and prediction button) on the right side.

4.User Management

Your personal details are displayed over the left side of the dashboard with an update

Info button to modify your personal details.

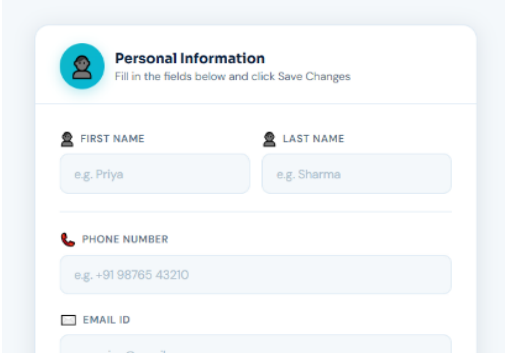


Figure 2.2.4: Update Info page

5. Dataset Module

You can upload a dataset which will get pre-processed, handles missing values and outliers and will later on train on a chosen model.

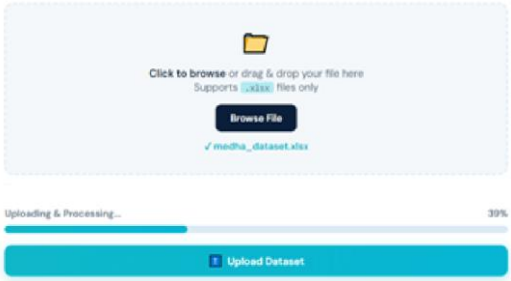


Figure 2.2.5: Upload dataset section with progress bar

6. Prediction Page

This is where you enter your valid input details and run the model and have a prediction generated which is now currently static.

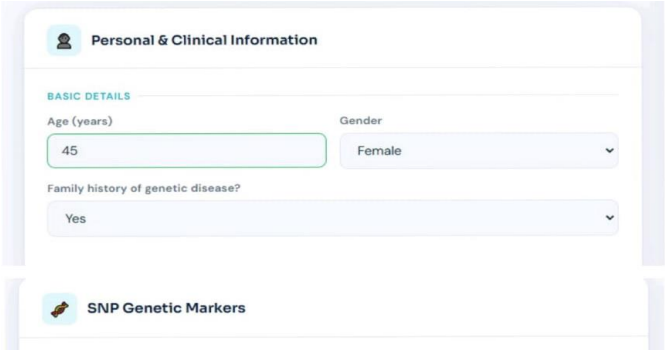


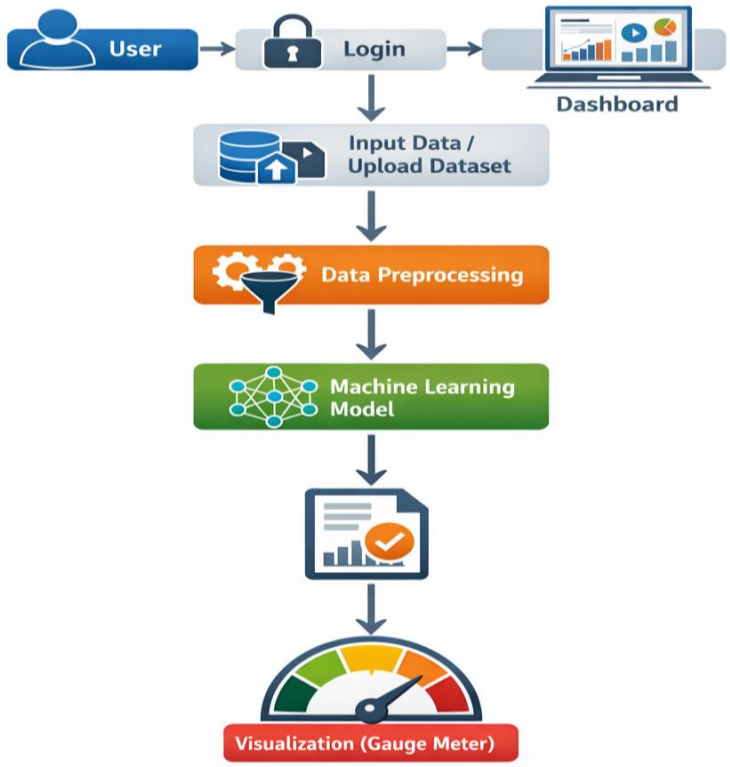
Figure 2.2.6: Input fields of Prediction Page

2.6 Output

- Risk Percentage
- Risk Category (Low / Medium / High)
- Visual Representation (Gauge Meter / Chart)

2.7 Diagram

Figure 2.2.7: System Flow



1. The process begins with the user accessing the system through the login interface. After successful authentication, the user is redirected to the dashboard, where they can view their personal details on the left and upload a dataset on the right
2. Once the dataset is uploaded, it is sent to the backend where pre-processing operations such as data cleaning, normalization, and encoding are performed. The processed dataset is then used for training and testing the machine learning model.
3. In the prediction phase, the user enters input parameters such as age, gender, SNP values, gene expression values, and family history. These inputs are validated and forwarded to the machine learning model.
4. The model processes the input data and generates a prediction indicating the risk level of a genetic disease. This output is then sent back to the front-end.
5. Finally, the result is displayed using a graphical visualization in the form of a risk gauge meter, this currently displays a static value of 45%.

2.8 Advantages

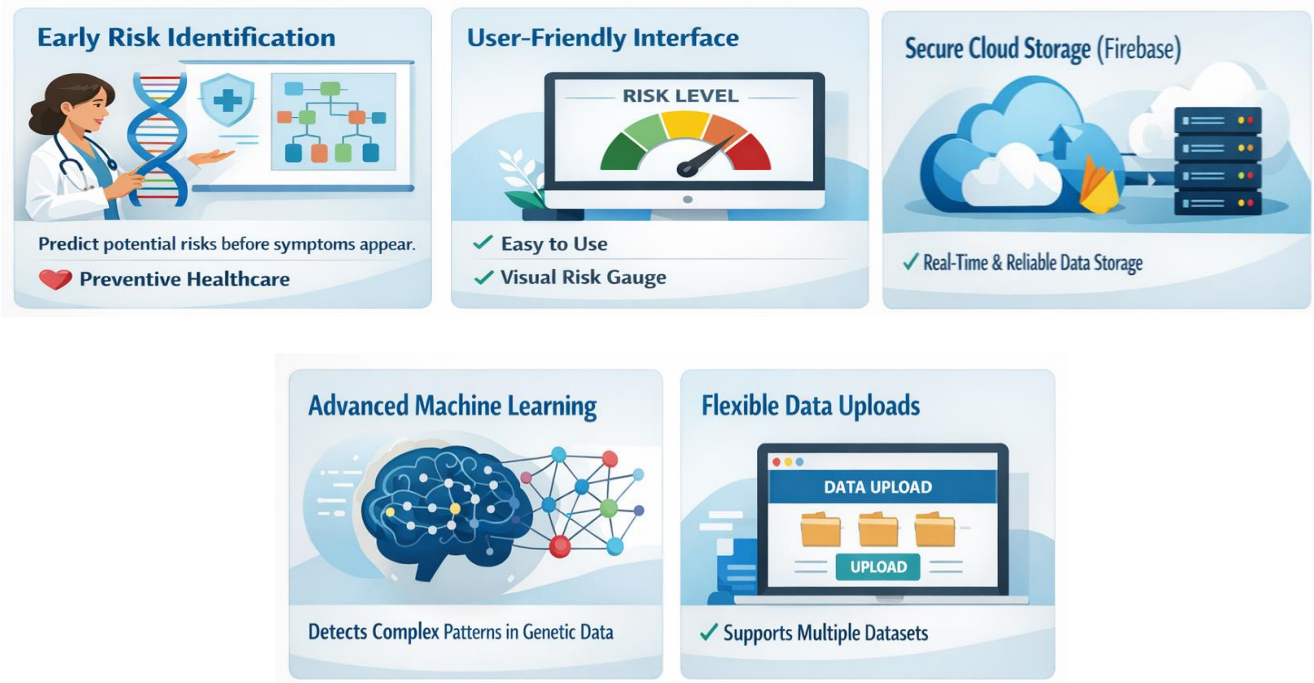


Figure 2.2.8: Key Advantages of the System

2.9 Limitations

Figure 2.2.9: Key Limitations of the System

2.10 Future Enhancements

The current system can be further enhanced by integrating advanced machine learning models to improve prediction accuracy.

Incorporating more genetic parameters and real world medical datasets can improve the reliability of predictions. The system can also be extended to include recommendation systems for preventive measures based on the predicted risk levels.

One major improvement would be the integration of a pre-trained machine learning model, eliminating the need for users to upload datasets manually. This would make the system more user-friendly and suitable for non-technical users.

Advanced machine learning algorithms such as **XGBoost**, **Random Forest** optimization, or **deep learning models** can be implemented to improve prediction accuracy.

The system can also be enhanced by integrating real-time medical datasets and APIs to provide more accurate and up-to-date predictions.

Another improvement would be the inclusion of additional input parameters such as lifestyle factors, environmental exposure, and medical history. Security features can also be strengthened to ensure complete protection of sensitive genetic and personal data.

Conclusion

In conclusion, the Genetic Disease Prediction System demonstrates the effective application of machine learning techniques in the healthcare domain. The system successfully integrates front-end, backend, database, and data processing components to provide a complete predictive solution.

By utilizing genetic indicators and family history, the system aims to identify potential disease risks at an early stage. Although the current implementation has certain limitations, it provides a strong foundation for future improvements.

With further enhancements and real-world validation, this system has the potential to contribute significantly to preventive healthcare and personalized medicine.